

Projekt: Software Engineering

Projektdokumentation

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

9. August 2026

Inhaltsverzeichnis

1	Projektübersicht	1
1.1	Zielgruppe und Kundennutzen	1
1.2	Funktionsumfang	1
1.3	Angestrebtes Ergebnis	1
2	Risikomanagement	2
3	MoSCoW-Priorisierung	2
4	Zeitplanung	3
4.1	Woche 1 – Konzeptionsphase, Teil 1 (27.07.–02.08.2026)	4
4.2	Woche 2 – Konzeptionsphase, Teil 2 (03.08.–09.08.2026)	4
4.3	Woche 3 – Erarbeitungs-/Reflexionsphase (10.08.–16.08.2026)	4
4.4	Woche 4 – Finalisierungsphase (17.08.–23.08.2026)	4
5	GitHub-Repository	4
6	Benutzeranleitung	5
	Literaturverzeichnis	5

1 Projektübersicht

Der folgende Abschnitt gibt einen Überblick zur Anwendung MietManager. Inhalt sind dabei Ausgangspunkt, Zielgruppe und Kundennutzen, geplanter Funktionsumfang sowie das mit dem Projekt angestrebte Ergebnis. MietManager ist eine unter Windows lauffähige Desktop-Anwendung (Python/PyQt6) zur Verwaltung von Immobilien, Mieteinheiten, Mietern und Mietverträgen für private Kleinvermieter mit wenigen Wohneinheiten. Kernprozess der Anwendung ist die automatisierte Erstellung einer Nebenkostenabrechnung.

1.1 Zielgruppe und Kundennutzen

Zielgruppe sind private Vermieter, die ihre Objekte bislang mit Excel oder auf Papier verwalten. Der zentrale Mehrwert gegenüber einer Tabellenkalkulation ist die automatische Berechnung der Nebenkostenabrechnung nach frei wählbarem Umlageschlüssel, wie zum Beispiel Fläche, Personen oder Anzahl Einheiten, je Kostenart und Mietvertrag sowie deren direkter Export als versandfertiger, personalisierter Geschäftsbrief mit Briefkopf, persönlicher Anrede, Kostenaufschlüsselung, Saldo und Grußformel. Die eigenen Vermieter-Stammdaten werden dazu einmalig in einem Profil hinterlegt und automatisch als Absender verwendet. Die erstellte Nebenkostenabrechnung kann anschließend manuell per E-Mail versendet oder ausgedruckt und postalisch verschickt werden.

1.2 Funktionsumfang

Den Kern der Anwendung bilden die Verwaltung von Immobilien sowie die Mieter- und Mietvertragsverwaltung, die automatisch auf zeitliche Überschneidungen je Mieteinheit prüft. Darauf aufbauend erfasst die Anwendung die Kostenpositionen einer Immobilie je Abrechnungszeitraum und berechnet daraus die Nebenkostenabrechnung, die als versandfertiger PDF-Brief exportiert werden kann. Das dazu benötigte Vermieter-Profil ist als Briefkopf-Absender konfigurierbar. Ergänzt wird die Anwendung durch ein Dashboard mit einer Kennzahlen-Übersicht zu Immobilien, Einheiten, Leerstand, aktiven Verträgen und Mieteinnahmen.

1.3 Angestrebtes Ergebnis

Eine funktionsfähige Desktop-Anwendung, die alle in der Anforderungserhebung definierten funktionalen und nichtfunktionalen Anforderungen erfüllt, mit realistischen Testdaten vorbefüllt ist und deren Kernprozess, die automatisierte Nebenkostenabrechnung, direkt nach dem Start nachvollziehbar ist.

2 Risikomanagement

Tabelle 2.1 fasst die identifizierten Projektrisiken zusammen und bewertet sie nach Eintrittswahrscheinlichkeit und Schwere der potenziellen Auswirkung. Als Gegenmaßnahme gegen die Planung von zu vielen Features im Bearbeitungszeitraum kommt dabei die MoSCoW-Priorisierung zum Einsatz, eine Methode aus dem Anforderungsmanagement, die Anforderungen in Muss (Must have), Soll (Should have), Kann (Could have) und Verzicht (Won't have) einteilt (Ahmad et al., 2017, S. 433). die konkrete Einteilung des Funktionsumfangs zeigt Tabelle 3.1 in Abschnitt 3.

#	Risiko	Wahrsch.	Auswirkung	Gegenmaßnahme
R1	Zu viele Features geplant	mittel	hoch	MoSCoW-Priorisierung
R2	Wartezeit auf Tutor-Feedback verzögert Zeitplan	hoch	hoch	Portfolioteile 1 und 2 möglichst früh statt am letzten Tag einreichen, um innerhalb der Woche noch auf Feedback reagieren zu können
R3	Berechnungslogik der Nebenkostenabrechnung unterschätzt	mittel	hoch	Rechenprototyp früh in Woche 3 bauen, bevor die GUI angebunden wird
R4	Packaging-Probleme	mittel	mittel	PyInstaller bereits in Woche 3 testen, nicht erst am Ende
R5	Zeitknappheit durch Studium/Beruf bei nur 4 Wochen Gesamtdauer	hoch	hoch	Klare Priorisierung nach MoSCoW, keine Pufferzeit für Nice-to-have-Features einplanen
R6	Datenverlust	niedrig	hoch	Bereits zu Beginn der Implementierung GitHub mit regelmäßigen, kleinteiligen Commits nutzen

Tabelle 2.1: Risikomanagement

3 MoSCoW-Priorisierung

Als Gegenmaßnahme gegen das in Abschnitt 2 identifizierte Risiko von zu vielen Features im Bearbeitungszeitraum wurde der Funktionsumfang nach der MoSCoW-Methode priorisiert. Die Muss-Anforderungen entsprechen den User Stories FA-01 bis FA-07 aus dem Anforderungsdokument. Tabelle 3.1 ordnet den gesamten betrachteten Funktionsumfang den vier Kategorien zu.

Kategorie	Umfang
Muss (Must have)	Immobilien- und Mieteinheitenverwaltung, Mieterverwaltung, Mietvertragsverwaltung mit Überlappungsprüfung, Kostenpositionserfassung, automatische Berechnung der Nebenkostenabrechnung, PDF-Export als versandfertiger Geschäftsbrief, Vermieterprofil als Briefkopf-Absender (FA-01 bis FA-07).
Soll (Should have)	Dashboard mit Übersicht zu Immobilien, Einheiten, Leerstand, aktiven Verträgen sowie Mietnahmen (FA-08).
Kann (Could have)	Automatischer E-Mail-Versand der Abrechnung direkt aus der Anwendung heraus, CSV-/Excel-Export der Kennzahlen.
Verzicht (Won't have)	Zahlungs- und Mahnwesen, Mehrbenutzerbetrieb, mobile Anwendung.

Tabelle 3.1: MoSCoW-Priorisierung des Funktionsumfangs

4 Zeitplanung

Das Projekt ist auf eine Gesamtdauer von vier Wochen angesetzt (Mo. 27.07.2026 bis So. 23.08.2026). Die Konzeptionsphase erhält davon zwei Kalenderwochen, da hier mit Projektdokumentation, Anforderungsdokument und Spezifikationsdokument die meisten Dokumente entstehen und eine sorgfältige Planung für den weiteren Projektverlauf entscheidend ist. Die Erarbeitungs- und Finalisierungsphase erhalten je eine Kalenderwoche. Trotz der zusätzlichen Woche bleibt der Puffer insgesamt knapp (siehe Risiken R2 und R5), weshalb die einzelnen Portfolioteile jeweils so früh wie möglich fertiggestellt werden, um noch Zeit für Tutor-Feedback und Korrekturen zu haben. Abbildung 4.1 zeigt den Projektzeitplan als Gantt-Diagramm. Anschließend werden die vier Wochen inhaltlich beschrieben. Der Projektfortschritt lässt sich an den drei Meilensteinen der Portfolioabgabe ablesen. Der Einreichung von Portfolioteil 1 (Konzeptionsphase) am 09.08.2026, von Portfolioteil 2 (Erarbeitungs-/Reflexionsphase) am 16.08.2026 und von Portfolioteil 3 (Finalisierungsphase) am 23.08.2026.

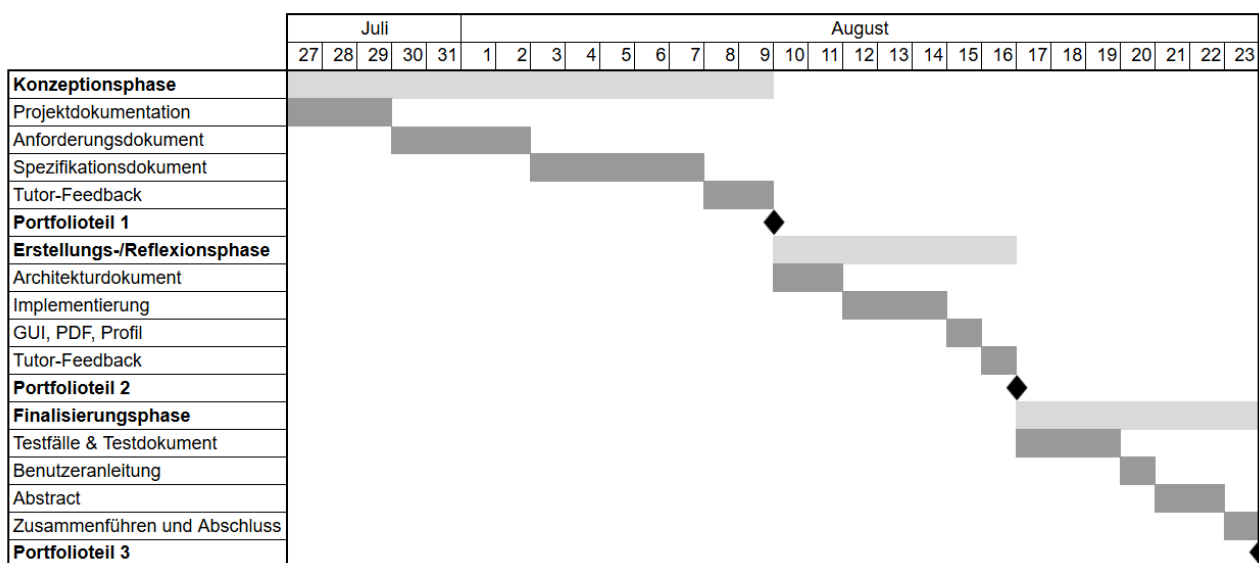


Abbildung 4.1: Projektzeitplan

4.1 Woche 1 – Konzeptionsphase, Teil 1 (27.07.–02.08.2026)

In der ersten Woche werden zunächst Projektübersicht, Risikomanagement, MoSCoW-Priorisierung und Zeitplanung als Grundlage der Projektdokumentation erstellt. Anschließend werden für das Anforderungsdokument die Stakeholder identifiziert, User Stories gesammelt, die nichtfunktionalen Anforderungen definiert und die funktionalen Anforderungen als UML-Use-Case-Diagramm visualisiert.

4.2 Woche 2 – Konzeptionsphase, Teil 2 (03.08.–09.08.2026)

In der zweiten Woche entsteht das Spezifikationsdokument. Das Datenmodell wird erarbeitet und als UML-Klassendiagramm visualisiert, der Kernprozess „Nebenkostenabrechnung erstellen“ als UML-Aktivitätsdiagramm dargestellt sowie Geschäftsregeln, Systemschnittstellen und Benutzerschnittstellen inklusive GUI-Mockups beschrieben. Vor Abgabe wird Tutor-Feedback zu allen drei Dokumenten der Konzeptionsphase eingeholt. Das Ergebnis der Konzeptionsphase ist die Einreichung von Portfolioteil 1.

4.3 Woche 3 – Erarbeitungs-/Reflexionsphase (10.08.–16.08.2026)

In der dritten Woche wird zunächst das Architekturdokument erstellt und die Struktur des GitHub-Repositorys aufgesetzt. Den Schwerpunkt der Woche bildet die Implementierung. Datenmodell und Datenbank, CRUD-Funktionen, die Berechnungslogik der Nebenkostenabrechnung, die grafische Oberfläche, der PDF-Export sowie das Vermieter-Profil werden umgesetzt. Auch hier wird vor Abgabe Tutor-Feedback eingeholt. Die Erarbeitungs- und Reflexionsphase endet mit der Einreichung von Portfolioteil 2.

4.4 Woche 4 – Finalisierungsphase (17.08.–23.08.2026)

Die vierte Woche beginnt mit der Einarbeitung des Tutor-Feedbacks aus Phase 2. Anschließend werden Testfälle geschrieben und im Testdokument protokolliert sowie die Benutzeranleitung verfasst. Zum Abschluss entsteht das zweiseitige Abstract, bevor alle Dokumente zusammengeführt und eine ZIP-Datei aus dem Inhalt des GitHub-Repositorys erzeugt werden. Die Finalisierungsphase signalisiert den Abschluss des Projekts und endet mit der Einreichung von Portfolioteil 3.

5 GitHub-Repository

(Ab Phase 2 wird hier der finale Link zum Github-Repository ergänzt.)

6 Benutzeranleitung

(Ab Phase 3 wird hier die Benutzeranleitung ergänzt.)

Literaturverzeichnis

Ahmad, K. S., Ahmad, N., Tahir, H., & Khan, S. (2017). Fuzzy_MoSCoW: A fuzzy based MoSCoW method for the prioritization of software requirements. *International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICICT)*, 433–437. <https://doi.org/10.1109/ICICICT1.2017.8342602>

Projekt: Software Engineering

Anforderungsdokument

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

9. August 2026

Inhaltsverzeichnis

1	Stakeholder	1
1.1	Vermieter (primärer Nutzer)	1
1.2	Mieter (indirekter Stakeholder)	1
1.3	Tutoren (im Rahmen des Kurses)	1
2	Funktionale Anforderungen	1
2.1	User Stories	1
2.2	UML-Use-Case-Diagramm	2
3	Nichtfunktionale Anforderungen	3
4	Glossar	3

1 Stakeholder

Dieser Abschnitt identifiziert die am Projekt beteiligten Ziel- und Benutzergruppen und beschreibt ihre jeweilige Rolle sowie ihr Interesse an der Anwendung.

1.1 Vermieter (primärer Nutzer)

Private Kleinvermieter mit wenigen Wohneinheiten sind die primäre Zielgruppe von MietManager. Sie bedienen die Anwendung direkt, verwalten ihre Immobilien, Mieter und Mietverträge und erstellen die Nebenkostenabrechnung. Diese Gruppe verwaltet ihre Objekte bislang häufig mit Excel oder auf Papier und hat in der Regel keine spezialisierte Software im Einsatz.

1.2 Mieter (indirekter Stakeholder)

Mieter interagieren nicht direkt mit der Anwendung, sind aber indirekt betroffen. Sie erhalten die von MietManager erzeugte Nebenkostenabrechnung als personalisierten Geschäftsbrief. Die Verständlichkeit und Korrektheit der Abrechnung wirkt sich damit unmittelbar auf diese Gruppe aus, auch wenn sie kein Benutzerkonto oder keinen Zugriff auf die Software besitzt.

1.3 Tutoren (im Rahmen des Kurses)

Im Rahmen dieses Portfoliokurses sind die betreuenden Tutoren gemäß Aufgabenstellung als Zielgruppe bzw. Benutzer der zu erstellenden Anwendung zu betrachten. Sie bewerten die Anwendung aus Sicht eines potenziellen Vermieters und geben Feedback zu Bedienbarkeit und Funktionsumfang.

2 Funktionale Anforderungen

Dieser Abschnitt beschreibt die funktionalen Anforderungen an MietManager zunächst als priorisierte User Stories und visualisiert sie anschließend in einem UML-Use-Case-Diagramm.

2.1 User Stories

Tabelle 2.1 listet die funktionalen Anforderungen als User Stories im Format „Als <Rolle> möchte ich <Ziel>, um <Nutzen> zu erreichen.“ auf und priorisiert sie nach der MoSCoW-Methode.

ID	User Story	Priorität
FA-01	Als Vermieter möchte ich Immobilien und deren Mieteinheiten anlegen und pflegen, um meinen Immobilienbestand zentral zu verwalten.	Muss
FA-02	Als Vermieter möchte ich Mieter mit ihren Kontaktdaten erfassen, um alle mieterbezogenen Informationen an einem Ort zu haben.	Muss
FA-03	Als Vermieter möchte ich Mietverträge anlegen, die automatisch auf zeitliche Überschneidungen je Mieteinheit geprüft werden, um Doppelvermietungen zu vermeiden.	Muss
FA-04	Als Vermieter möchte ich Kostenpositionen je Immobilie und Zeitraum erfassen, um die Grundlage für die Nebenkostenabrechnung zu schaffen.	Muss
FA-05	Als Vermieter möchte ich die Nebenkostenabrechnung automatisch nach einem frei wählbaren Umlageschlüssel berechnen lassen, um Zeit zu sparen und Rechenfehler zu vermeiden.	Muss
FA-06	Als Vermieter möchte ich die berechnete Nebenkostenabrechnung als versandfertigen, personalisierten PDF-Brief exportieren, um sie ohne Zwischenschritt an meine Mieter zu verschicken.	Muss
FA-07	Als Vermieter möchte ich meine eigenen Stammdaten einmalig in einem Profil hinterlegen, damit sie automatisch als Briefkopf verwendet werden.	Muss
FA-08	Als Vermieter möchte ich ein Dashboard mit den wichtigsten Kennzahlen sehen, um schnell einen Überblick über Leerstand und Mieteinnahmen zu erhalten.	Soll

Tabelle 2.1: User Stories

2.2 UML-Use-Case-Diagramm

Abbildung 2.1 visualisiert die funktionalen Anforderungen aus Sicht des Akteurs „Vermieter“. Da Mieter nicht direkt mit der Anwendung interagieren, treten sie im Use-Case-Diagramm nicht als Akteur auf (siehe Abschnitt 1.2).

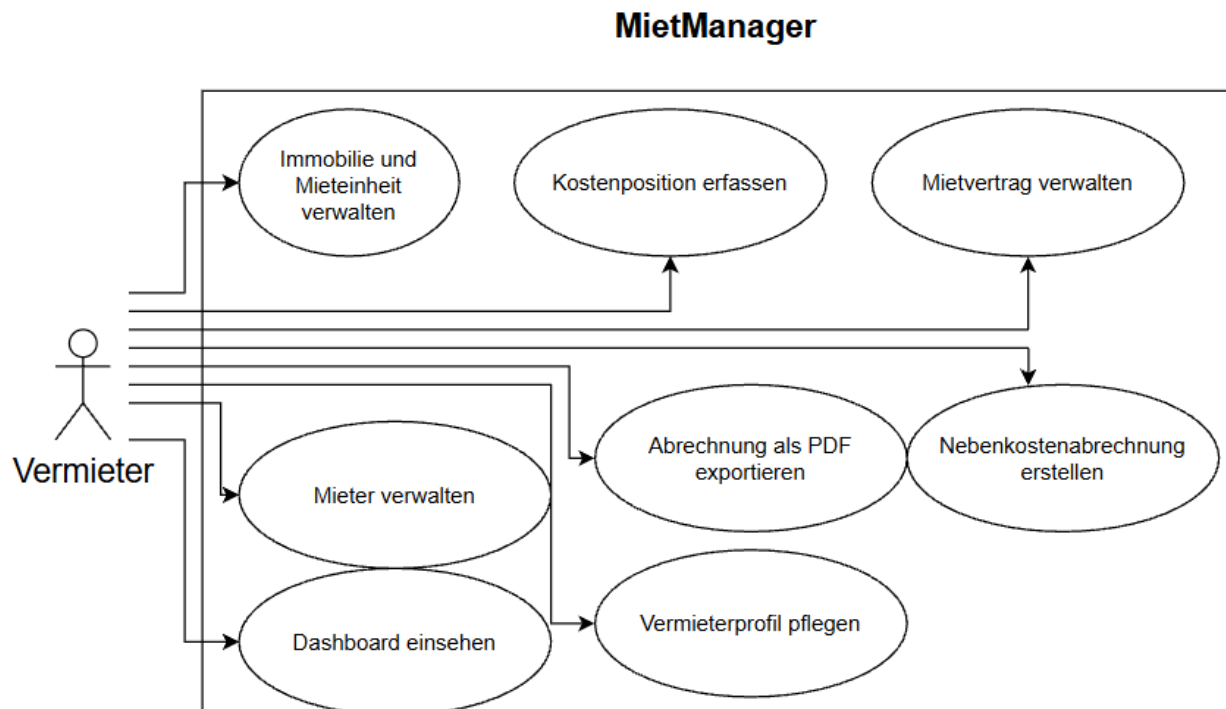


Abbildung 2.1: UML-Use-Case-Diagramm

3 Nichtfunktionale Anforderungen

Im Hinblick auf die Benutzerfreundlichkeit ist die Bedienoberfläche selbsterklärend und ohne Schulung nutzbar zu gestalten. Eingaben werden validiert, und Fehlermeldungen benennen konkret, was zu korrigieren ist. Bezüglich der Performance reagiert die Benutzeroberfläche bei einem Bestand von bis zu 50 Mieteinheiten innerhalb von 200 ms auf Benutzerinteraktionen. Für die Datensicherheit werden die Daten ausschließlich lokal in einer SQLite-Datenbank im Benutzerprofil gespeichert. Eine Übertragung an Dritte findet nicht statt. Hinsichtlich der Portabilität ist die Anwendung unter Windows 10/11 lauffähig, ohne dass über die vom Kurs zugelassenen Standard-Laufzeitumgebungen (Python-Interpreter bzw. eigenständige .exe) hinaus zusätzliche Software installiert werden muss.

4 Glossar

Das folgende Glossar erläutert die im Projekt verwendeten Fachbegriffe und Abkürzungen.

Begriff	Erläuterung
Immobilie	Ein vermietetes Gebäude oder Grundstück, bestehend aus einer oder mehreren Mieteinheiten.
Mieteinheit	Einzelne vermietbare Einheit innerhalb einer Immobilie.
Umlageschlüssel	Regel, nach der die Kosten einer Kostenposition auf die Mieteinheiten einer Immobilie verteilt werden z.B. nach Fläche, Personenzahl oder Anzahl Einheiten.
Kostenposition	Eine einzelne erfasste Betriebskostenart einer Immobilie für einen bestimmten Zeitraum.
Nebenkostenabrechnung	Das aus den Kostenpositionen eines Zeitraums berechnete Ergebnis je Mietvertrag.
Vermieterprofil	Die Stammdaten des Vermieters, die als Absender auf der Nebenkostenabrechnung erscheinen.
MoSCoW-Priorisierung	Methode aus dem Anforderungsmanagement, die Anforderungen in Muss (Must have), Soll (Should have), Kann (Could have) und Verzicht (Won't have) einteilt.

Tabelle 4.1: Glossar

Projekt: Software Engineering

Spezifikationsdokument

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

9. August 2026

Inhaltsverzeichnis

1	Datenmodell	1
1.1	Geschäftsobjekte	1
1.2	UML-Klassendiagramm	1
2	Geschäftsprozesse	2
2.1	Kernprozess: Nebenkostenabrechnung erstellen	2
3	Geschäftsregeln	3
4	Systemschnittstellen	4
5	Benutzerschnittstellen	4
5.1	Struktur	4
5.2	Dialogflüsse und Eingabevalidierung	4
5.3	GUI-Mockups	5

1 Datenmodell

Dieser Abschnitt beschreibt die zentralen Geschäftsobjekte von MietManager und ihre Beziehungen zueinander, zunächst textuell und anschließend als UML-Klassendiagramm.

1.1 Geschäftsobjekte

MietManager verwaltet als zentrales Geschäftsobjekt die Immobilie. Dabei handelt es sich um ein vermietetes Gebäude, identifiziert durch Bezeichnung und Adresse, das beliebig viele Mieteinheiten, Kostenpositionen und Nebenkostenabrechnungen besitzt. Eine Mieteinheit ist eine einzelne vermietbare Wohnung innerhalb einer Immobilie, deren Fläche und Zimmeranzahl als Grundlage für die Umlage von Kosten dienen. Ein Mieter ist eine natürliche Person mit Kontaktdaten, die einen oder mehrere Mietverträge abschließen kann. Ein Mietvertrag verknüpft einen Mieter mit einer Mieteinheit für einen Zeitraum und legt Kaltmiete, Nebenkostenvorauszahlung, Kautions- und Personenzahl fest.

Eine Kostenposition ist eine einzelne Betriebskostenart einer Immobilie für einen Erfassungszeitraum, inklusive des zu verwendenden Umlageschlüssels. Die Nebenkostenabrechnung ist das Ergebnis der Abrechnungsberechnung einer Immobilie für einen Zeitraum und besteht aus einer Abrechnungsposition je betroffenem Mietvertrag, die Kostenanteil, geleistete Vorauszahlung und Saldo enthält. Das Vermieterprofil bündelt die Stammdaten des Vermieters, wobei es ein eigenständiges Objekt ohne Beziehung zu den übrigen Geschäftsobjekten ist und als Briefkopf-Absender beim PDF-Export dient.

1.2 UML-Klassendiagramm

Abbildung 1.1 visualisiert die Geschäftsobjekte und ihre Beziehungen. Kompositionsbeziehungen (gefüllte Raute) bedeuten, dass das untergeordnete Objekt nicht ohne das übergeordnete existieren kann. Einfache Assoziationen (Linie) verknüpfen unabhängig existierende Objekte.

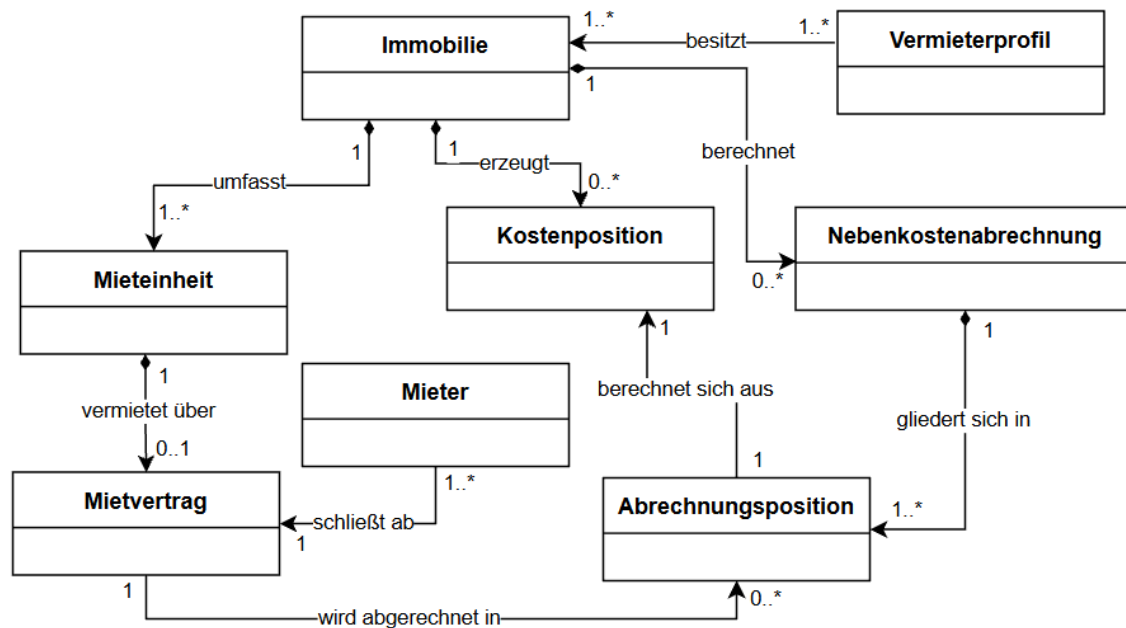


Abbildung 1.1: UML-Klassendiagramm

2 Geschäftsprozesse

Dieser Abschnitt beschreibt den zentralen Geschäftsprozess von MietManager textuell und visualisiert ihn anschließend als UML-Aktivitätsdiagramm.

2.1 Kernprozess: Nebenkostenabrechnung erstellen

Der zentrale Geschäftsprozess von MietManager berechnet die Nebenkostenabrechnung einer Immobilie für einen gewählten Zeitraum. Zunächst wählt der Vermieter eine Immobilie sowie Start- und Enddatum des Abrechnungszeitraums. Das System lädt daraufhin alle Kostenpositionen der Immobilie, die den Zeitraum überschneiden, und ermittelt alle im Zeitraum aktiven Mietverträge. Liegen keine Kostenpositionen oder keine aktiven Mietverträge vor, bricht das System mit einer Fehlermeldung ab. Andernfalls berechnet das System für jede Kostenposition je aktivem Mietvertrag den Anteil nach dem hinterlegten Umlageschlüssel und summiert die Anteile je Mietvertrag auf. Die Umlageschlüssel sind dabei Fläche, Personenzahl oder Anzahl Einheiten. Anschließend stellt das System den aufsummierten Kostenanteil der geleisteten Nebenkostenvorauszahlung gegenüber und berechnet den Saldo. Ist der Saldo positiv, wird er als Guthaben identifiziert, andernfalls als Nachzahlung. Das Ergebnis wird als Nebenkostenabrechnung mit einer Abrechnungsposition je Mietvertrag gespeichert, sodass der Vermieter die Abrechnung abschließend als personalisierten PDF-Brief je Mieter exportieren kann. Die Stammdaten des Vermieters werden dabei als Briefkopf-Absender verwendet und können im Profil gepflegt werden. Abbildung 2.1 visualisiert den beschriebenen Kernprozess.

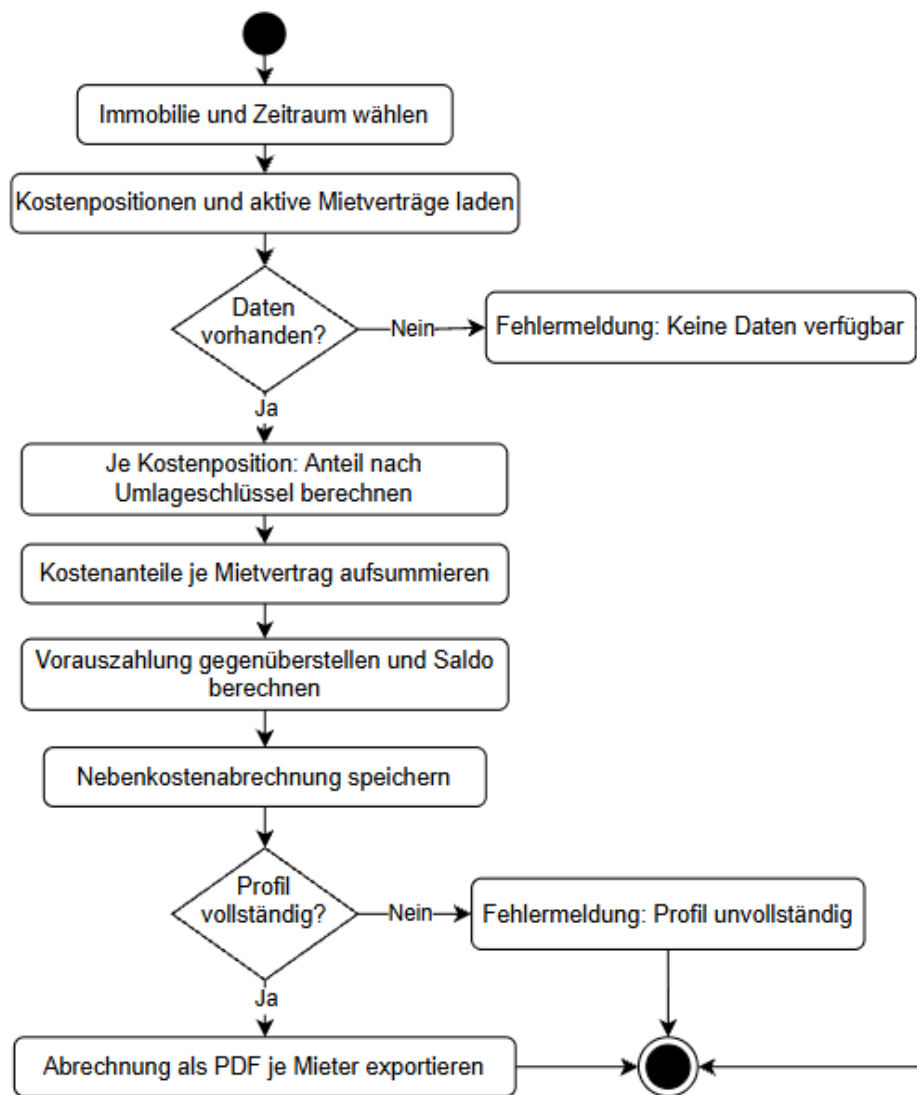


Abbildung 2.1: UML-Aktivitätsdiagramm: Nebenkostenabrechnung erstellen

3 Geschäftsregeln

Für Mietverträge gilt, dass das Startdatum vor dessen Enddatum liegen muss, sofern ein Enddatum gesetzt ist. Unbefristete Verträge haben kein Enddatum. Zudem darf eine Mieteinheit im selben Zeitraum nicht durch mehr als einen Mietvertrag belegt sein, sodass aktive Verträge derselben Mieteinheit sich zeitlich nicht überlappen dürfen.

Beim Löschen von Datensätzen greifen weitere Regeln. Eine Immobilie kann nur gelöscht werden, wenn keine ihrer Mieteinheiten einen bestehenden Mietvertrag hat. Eine Mieteinheit kann außerdem nur gelöscht werden, wenn kein Mietvertrag auf sie verweist. Ein Mieter kann nur gelöscht werden, wenn ihm kein Mietvertrag mehr zugeordnet ist.

Für die Nebenkostenabrechnung gilt schließlich, dass der Start eines Abrechnungszeitraums vor dessen Ende liegen muss und eine Abrechnung nur erstellt werden kann, wenn für den Zeitraum mindes-

tens eine Kostenposition und mindestens ein aktiver Mietvertrag der gewählten Immobilie vorliegen. Für den anschließenden PDF-Export müssen im Vermieterprofil mindestens Name, Straße, Postleitzahl und Ort hinterlegt sein.

4 Systemschnittstellen

MietManager ist eine lokale Desktop-Anwendung ohne Anbindung an externe Systeme oder Netzwerkdienste. Die einzige technische Schnittstelle nach außen ist der Dateisystemzugriff. Zur Datenehaltung dient eine lokale SQLite-Datenbankdatei im Benutzerprofil des Betriebssystems, und beim PDF-Export erzeugt die Anwendung PDF-Dateien im vom Benutzer gewählten Zielverzeichnis des Dateisystems.

5 Benutzerschnittstellen

Dieser Abschnitt beschreibt die Struktur der grafischen Benutzeroberfläche von MietManager sowie die Dialogflüsse und die Eingabevalidierung bei der Datenerfassung.

5.1 Struktur

Die Anwendung besteht aus einem Hauptfenster mit tabellarischer Navigation und einem Profil-Popup. Das Dashboard zeigt eine Kennzahlen-Übersicht für Immobilien, Mieteinheiten, Leerstand, aktive Verträge und Mieteinnahmen als Karten. Im Tab Immobilien werden die Immobilien mit ihren Mieteinheiten in einer Baumansicht dargestellt, inklusive Anlegen, Bearbeiten und Löschen. Die Tabs Mieter, Mietverträge und Kostenpositionen bieten jeweils eine tabellarische Verwaltung der entsprechenden Datensätze. Der Tab Nebenkostenabrechnung enthält ein Formular zur Auswahl von Immobilie und Zeitraum, eine tabellarische Ergebnisanzeige sowie den PDF-Export. Über den Profil-Button oben rechts öffnet sich das Profil-Popup mit dem Formular zur Pflege der Vermieter-Stammdaten.

5.2 GUI-Mockups

Abbildung 5.1 zeigt handskizzierte Mockups der zentralen Ansichten Dashboard, tabellarischer Daten-Tab, stellvertretend für Immobilien, Mieter, Mietverträge und Kostenpositionen, Tab Nebenkostenabrechnung sowie das Profil-Popup.

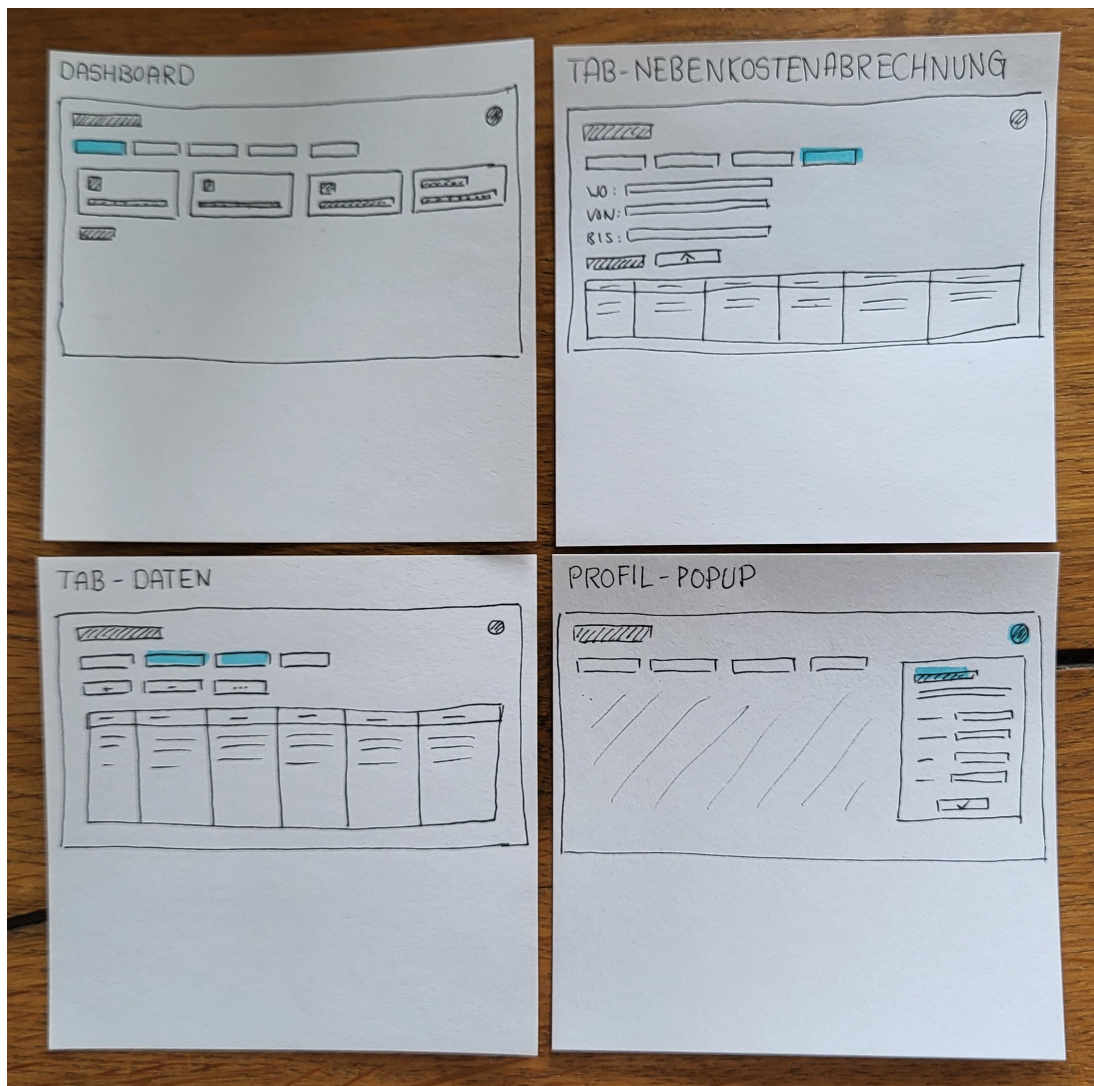


Abbildung 5.1: Handskizzierte GUI-Mockups der zentralen Ansichten